

Voxel Royale Distribuído: um esqueleto de *battle royale* em tempo real com gRPC e *web services*

Grupo de Computação Distribuída — PUC Minas

Aluno 1 PLACEHOLDER, Aluno 2 PLACEHOLDER, Aluno 3 PLACEHOLDER,
Aluno 4 PLACEHOLDER, Aluno 5 PLACEHOLDER, Aluno 6 PLACEHOLDER,
Aluno 7 PLACEHOLDER, Aluno 8 PLACEHOLDER, Aluno 9 PLACEHOLDER

Resumo

Resumo. Este relatório descreve a primeira entrega do Voxel Royale Distribuído, um mini *battle royale* 2D projetado para rodar no navegador do celular. O foco da Entrega 1 é demonstrar uma arquitetura distribuída clara que satisfaça, obrigatoriamente, dois requisitos do enunciado: comunicação interna via **gRPC/RPC** e exposição de ***web services*** HTTP. Apresentamos a divisão em três microsserviços Go — Gateway, Lobby e Game —, os contratos *Protocol Buffers*, o fluxo de mensagens e o empacotamento reproduzível em Docker Compose.

Abstract. This report describes the first delivery of Voxel Royale Distribuído, a mobile-first 2D battle royale. The goal of this delivery is to demonstrate a clear distributed architecture satisfying two mandatory course requirements: internal **gRPC/RPC** communication and HTTP ***web services***. We present three Go microservices (Gateway, Lobby and Game), the Protocol Buffers contracts, the message flow and a reproducible Docker Compose packaging.

1 Problema

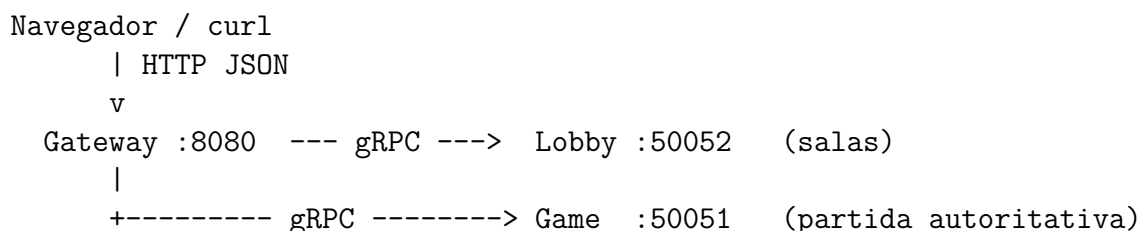
O objetivo do trabalho é construir, de forma jogável e mensurável, um sistema distribuído em tempo real no qual até 50 jogadores participam de uma partida *battle royale* voxel. Os jogadores entram por QR Code, informam apenas um nome (sem conta ou autenticação) e disputam partidas rápidas de até cinco minutos. O desafio central, do ponto de vista de Computação Distribuída, não é o jogo em si, mas evidenciar conceitos da disciplina: comunicação entre processos, contratos tipados, concorrência, um servidor autoritativo sobre o estado e um caminho claro para observabilidade, tolerância a falhas e teste de carga nas próximas fases.

A Entrega 1 exige a implementação de pelo menos dois requisitos do enunciado. O grupo escolheu **gRPC/RPC** para a comunicação interna entre serviços e ***web ser-***

vices HTTP para a operação e a entrada de jogadores. Esta entrega entrega um *esqueleto distribuído* executável que prova esses dois requisitos ponta a ponta.

2 Arquitetura

O sistema é um monorepo Go dividido em três microsserviços que se comunicam por gRPC e são orquestrados por Docker Compose. O **Gateway** é a borda HTTP pública; o **Lobby** gerencia o ciclo de vida das salas; e o **Game** é o servidor autoritativo da partida.



O Gateway traduz requisições HTTP/JSON em chamadas gRPC usando *grpc-gateway*: as anotações `google.api.http` nos arquivos `.proto` definem cada rota REST a partir de um método RPC. Assim, o mesmo contrato *Protocol Buffers* serve tanto para a comunicação interna tipada (gRPC) quanto para a API pública (web service), evitando divergência entre as duas interfaces. Os serviços se descobrem por DNS interno do Compose (`game:50051`, `lobby:50052`) e cada um expõe um `/healthz` usado pelos *healthchecks*.

3 Requisitos implementados

gRPC/RPC (requisito 1). Os contratos vivem em `proto/lobby/v1/lobby.proto` e `proto/match/v1/match.proto`, dos quais geramos o código Go com `protoc`. O Lobby implementa o serviço `LobbyService` com seis RPCs: `CreateRoom`, `JoinRoom`, `GetRoom`, `StartRoom`, `LeaveRoom` e `SetReady`. O Game implementa `GameService.StreamMatch`, que recebe o *input* do jogador e devolve um *snapshot* autoritativo da partida (posições, vida, armas, baús, zona segura e ranking).

Web services HTTP (requisito 2). O Gateway publica, na porta 8080, web services REST/JSON mapeados diretamente dos RPCs: `POST /v1/rooms`, `POST /v1/rooms/{id}/join`, `GET /v1/rooms/{id}`, `POST /v1/rooms/{id}/start`, `POST /v1/rooms/{id}/leave`, `POST /v1/rooms/{id}/ready` e `POST /v1/match/stream`, além de `GET /healthz`. A referência completa de mensagens está em `docs/messages.md`.

4 Detalhes de implementação

O backend usa Go 1.25 e `google.golang.org/grpc`. Cada serviço é um processo independente em `services/` com lógica isolada em `internal/`, o que permite testes

unitários por serviço (`go test ./...` cobre Gateway, Lobby e Game). O estado é mantido em memória: o Lobby guarda salas e jogadores com controle de concorrência por *mutex*; o Game mantém uma partida autoritativa que valida o *input*, descarta entradas repetidas por *input_sequence*, avança um *tick* e recalcula movimento (limitado), abertura de baús, dano por arma, zona segura que encolhe e ranking ordenado.

O empacotamento usa um `Dockerfile` por serviço e um `deployments/docker-compose.yml` no qual o Gateway só sobe após Game e Lobby ficarem saudáveis (`depends_on` com `condition: service_healthy`). Isso torna a demonstração reproduzível em qualquer máquina com Docker, com um caminho direto para implantação em VPS.

5 Desafios

O principal desafio foi consolidar uma base que havia sido revertida do `master` e estava espalhada em *branches*, transformando-a em um monorepo coerente com fronteiras claras entre contrato, serviço e infraestrutura. Decidir que o código gerado ficaria em `gen/` e os contratos versionados em `proto/<svc>/v1` estabilizou os *imports*. Outro ponto foi a coordenação do grupo de nove pessoas: definimos *ownership* por squad e um processo de mudança de contrato para evitar quebras. Ficam para as próximas fases conectar `Lobby.StartRoom` ao Game, adicionar o transporte WebSocket em tempo real, logs correlacionados e a prova de carga com 50 jogadores.

6 Papéis dos alunos

O grupo se organiza em três squads por fatia funcional (detalhe em `docs/roles.md`):

- **Squad A** — fluxo canônico de início de partida (Lobby + Game).
- **Squad B** — contrato e esqueleto distribuído (Gateway, `.proto`, geração).
- **Squad C** — demo, empacotamento e relatório (Docker Compose, smoke test, SBC).

Cada squad é responsável por escrever a seção do relatório correspondente ao que implementou. Os nomes dos nove alunos e os *owners* de cada squad estão marcados como `PLACEHOLDER` e devem ser preenchidos antes da submissão.

7 Conclusão

A Entrega 1 cumpre seu objetivo: um esqueleto distribuído executável que demonstra, de forma verificável, comunicação interna por gRPC e *web services* HTTP, com documentação de arquitetura e mensagens. A base está pronta para evoluir para o pipeline em tempo real, a jogabilidade completa e a observabilidade exigida na Entrega 2.

Referências

- [1] gRPC Authors. *gRPC: A high performance, open source universal RPC framework*. <https://grpc.io>.
- [2] gRPC-Gateway Authors. *grpc-gateway: gRPC to JSON proxy generator*. <https://github.com/grpc-ecosystem/grpc-gateway>.
- [3] Google. *Protocol Buffers*. <https://protobuf.dev>.